

ASYMMETRIC CRYPTOGRAPHY AND KEY MANAGEMENT

Understand public-key cryptography:

Public-key cryptography (*also called asymmetric cryptography*) is a cryptographic system that uses two different keys instead of one shared key.

These keys are mathematically related but serve different purposes.

- **Public Key** – shared openly with others
- **Private Key** – kept secret by the owner

Basically, it is a system where one key locks the message and the other key unlocks it, solving many problems of secure communication.

Why It Exists?

Before asymmetric cryptography, systems used **symmetric encryption**, where both sender and receiver needed the **same secret key**.

- **The problem:** securely sharing that key in the first place.

Asymmetric cryptography solves this by allowing anyone to encrypt messages using the public key, while only the owner can decrypt using the private key.

Public and Private Keys

- **PUBLIC KEY** — is openly distributed and can be shared with anyone.

It is used to:

- Encrypt messages sent to the key owner
- Verify digital signatures created by the private key

Example workflow:

1. Alice publishes her public key.
2. Bob encrypts a message using Alice's public key.
3. Only Alice can decrypt it with her private key.

Basically, it is the lock that everyone can use, but only the owner has the key to open it.

- **PRIVATE KEY** — must always remain secret.

It is used to:

- Decrypt messages encrypted with the public key
- Create digital signatures

- **If the private key is compromised, the entire cryptographic system becomes insecure.**

Basically, it is the master key that proves identity and unlocks encrypted data.

THE KEY DISTRIBUTION PROBLEM:

— refers to the difficulty of securely sharing encryption keys between parties. *In symmetric cryptography, both parties must know the same secret key beforehand.*

This creates problems such as:

- Secure transmission of the key
- Risk of interception
- Scalability issues with many users

Example:

If 100 people communicate securely using symmetric encryption, they might need **4950 unique keys**. Asymmetric cryptography reduces this complexity. *Basically, it is the problem of how to safely give someone a secret key without anyone else seeing it.*

Public-key cryptography solves this by allowing users to share **public keys freely**.

RSA CRYPTOGRAPHY (Rivest-Shamir-Adleman Algorithm)

— is one of the most widely used public-key cryptosystems.

- It was invented in 1977 by Ron Rivest, Adi Shamir, and Leonard Adleman.
- It is secure because **multiplying two primes is easy, but reversing it (factoring) is extremely difficult.**

RSA Algorithm:

RSA Algorithm is based on **factorization** of large number and **modular arithmetic** for encrypting and decrypting data. It consists of **three main stages**:

1. **Key Generation:** Creating Public and Private Keys
2. **Encryption:** Sender encrypts the data using Public Key to get cipher text.
3. **Decryption:** Decrypting the cipher text using Private Key to get the original data.

1. Key Generation

- Choose two large prime numbers, say **p** and **q**. These prime numbers should be kept secret.
- Calculate the product of primes, **n = p * q**. This product is part of the public as well as the private key.
- Calculate Euler Totient Function $\Phi(n)$ as $\Phi(n) = \Phi(p * q) = \Phi(p) * \Phi(q) = (p - 1) * (q - 1)$.
- Choose encryption exponent **e**, such that
 - $1 < e < \Phi(n)$, and
 - $\gcd(e, \Phi(n)) = 1$, that is e should be co-prime with $\Phi(n)$.
- Calculate decryption exponent **d**, such that
 - $(d * e) \equiv 1 \pmod{\Phi(n)}$, that is d is modular multiplicative inverse of $e \pmod{\Phi(n)}$. Some common methods to calculate multiplicative inverse are: Extended Euclidean Algorithm, Fermat's Little Theorem, etc.
 - We can have multiple values of d satisfying $(d * e) \equiv 1 \pmod{\Phi(n)}$ but it does not matter which value we choose as all of them are valid keys and will result into same message on decryption.

Finally, the **Public Key** = (n, e) and the **Private Key** = (n, d).

2. Encryption

To encrypt a message **M**, it is first converted to numerical representation using ASCII and other encoding schemes. Now, use the public key (n, e) to encrypt the message and get the cipher text using the formula:

$$C = M^e \pmod n, \text{ where } C \text{ is the Cipher text and } e \text{ and } n \text{ are parts of public key.}$$

3. Decryption

To decrypt the cipher text **C**, use the private key (n, d) and get the original data using the formula:

$$M = C^d \pmod n, \text{ where } M \text{ is the message and } d \text{ and } n \text{ are parts of private key.}$$

Idea behind RSA Algorithm:

The idea of RSA is based on the fact that it is difficult to factorize a large integer. The Public Key is (n, e), where n and e are publicly known, while the Private Key is (n, d). Since only the receiver knows the value of d, only they can decrypt the message. **But is it possible to find the value of d using n and e?**

We know that $(d * e) \equiv 1 \pmod{\Phi(n)}$, so if we can calculate the value of $\Phi(n)$, we can find the value of d. But $\Phi(n) = (p - 1) * (q - 1)$. So, we need the value of p and q. Now, one might think that it's quite easy to find the value of p and q as $n = p * q$ and n is already publicly known but RSA Algorithm takes the value of p and q to be very large which in turn makes the value of n extremely large and factorizing such a large value is computationally impossible.

Therefore encryption strength lies in the values of **p** and **q**. RSA keys can be typically 1024 or 2048 bits long, but experts believe that 1024-bit keys could be broken shortly. But till now it seems to be an infeasible task.

Note: If someone gets to know the value of **p** and **q**, then he can calculate the value of **d** and decrypt the message.

Mathematical Principles of RSA:

- **Modular Arithmetic**

- RSA relies heavily on modular arithmetic, which works with remainders.
- EXAMPLE: $17 \bmod 5 = 2$
- Modular arithmetic keeps numbers manageable even when they become extremely large.
- Basically, it is arithmetic where numbers wrap around after reaching a certain value (the modulus).

- **Prime Numbers**

- Prime numbers are numbers that have only two divisors: 1 and itself.
- EXAMPLES: 2, 3, 5, 7, 11, 13
- RSA uses two very large prime numbers to generate keys.
- The security of RSA relies on the difficulty of factoring the product of two large primes.
- Basically, it is secure because finding the original primes from their multiplied result is computationally very hard.

RSA DIGITAL SIGNATURES:

What is Digital Signature?

— authenticate the identity of the sender and guarantee the integrity of the message. By using a private key to create a unique signature and a public key to verify it, digital signatures ensure that messages are not tampered with during transmission. This technology is vital for ensuring trust and security in various online transactions and communications, making it an indispensable tool in modern cybersecurity.

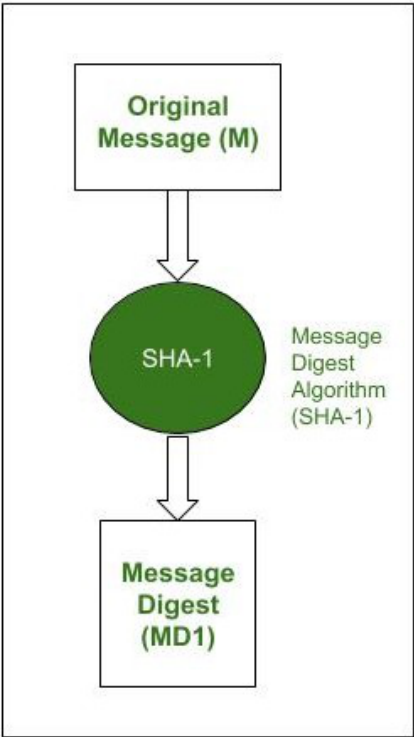
RSA can also provide authentication and integrity.
Instead of encrypting with the public key, the sender uses the private key.

RSA Signature Scheme:

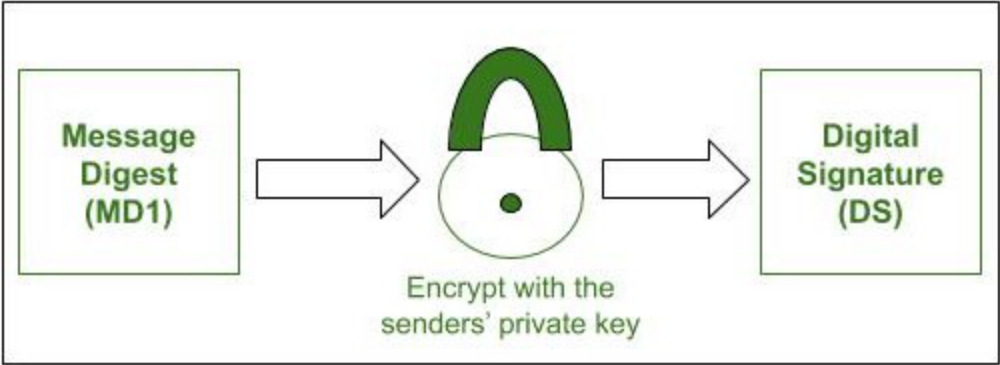
Assume that there is a sender (A) and a receiver (B). A wants to send a message (M) to B along with the digital signature (DS) calculated over the message.

STEPS:

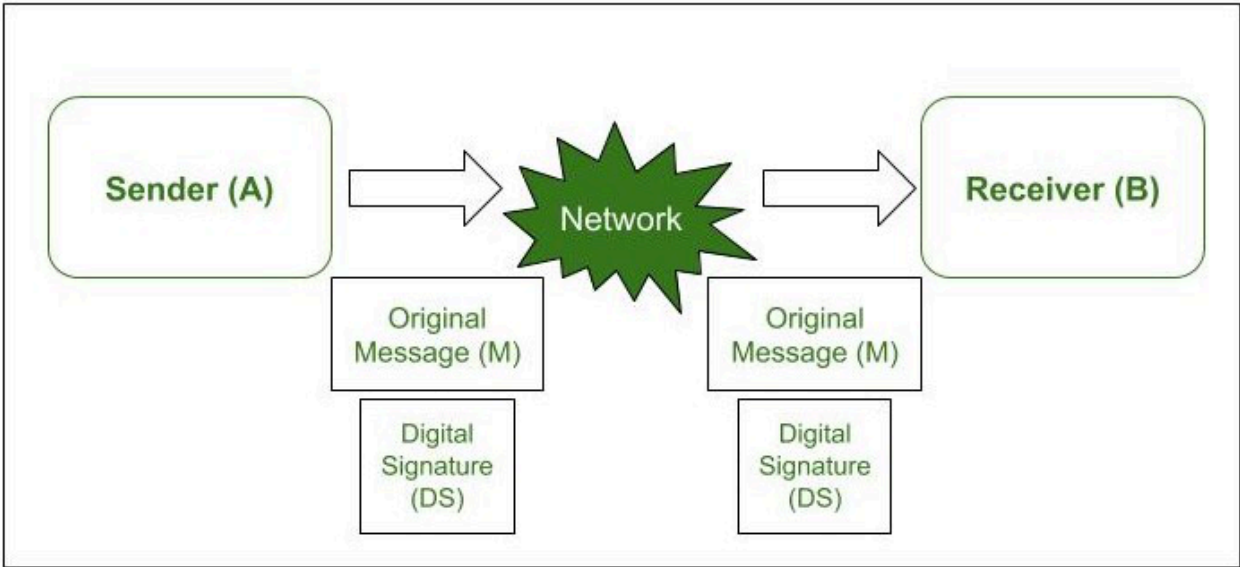
1. Sender A uses the SHA-1 Message Digest Algorithm to calculate the message digest (MD1) over the original message M.



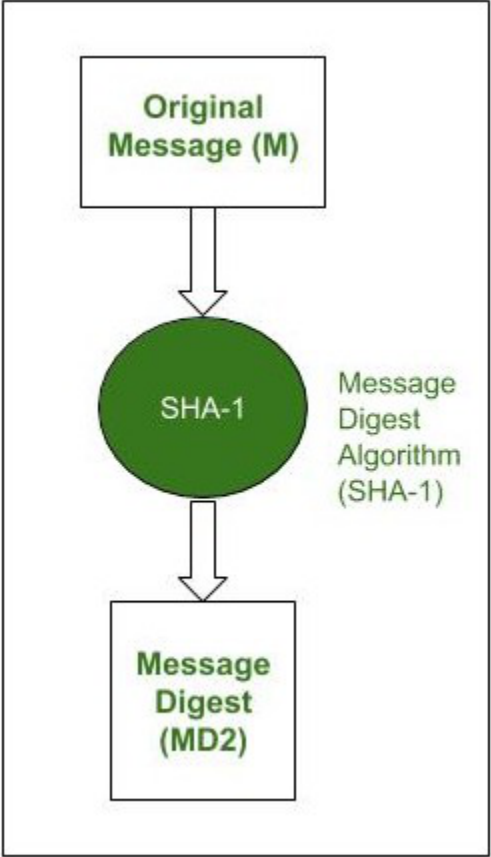
2. A now encrypts the message digest with its private key. The output of this process is called Digital Signature (DS) of A.



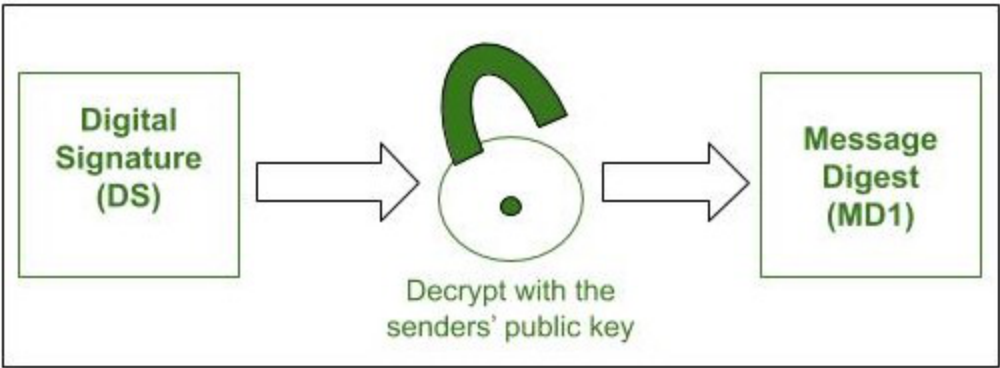
3. Now sender A sends the digital signature (DS) along with the original message (M) to B.



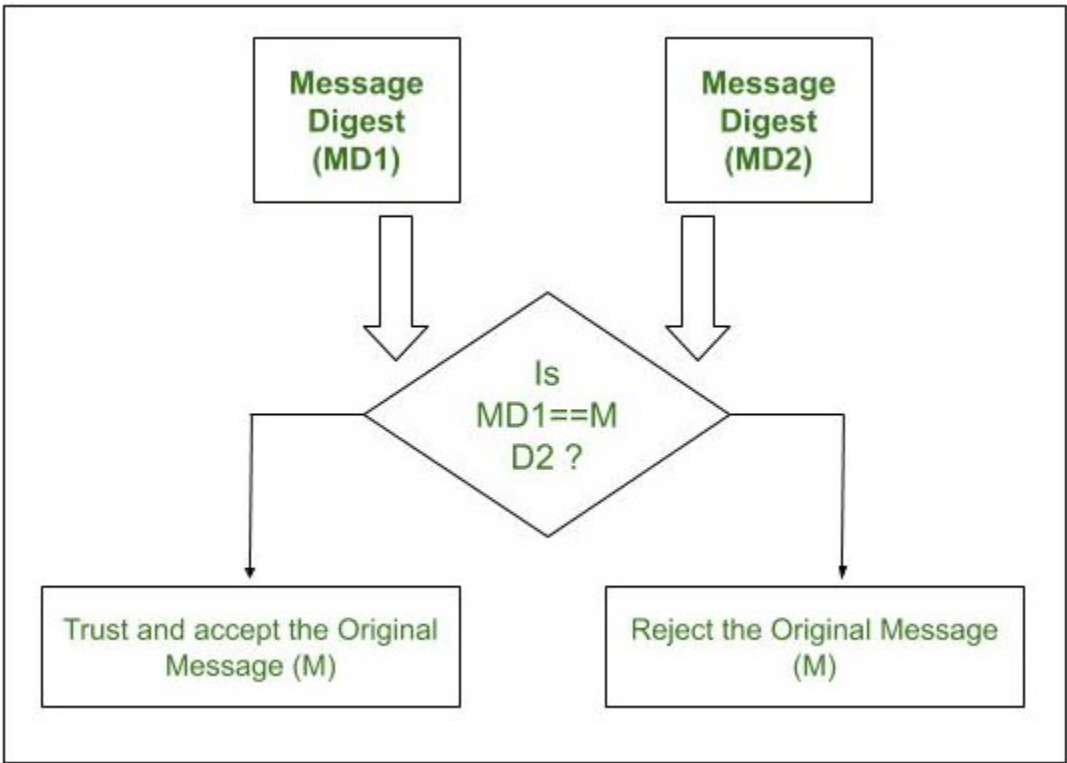
4. When B receives the Original Message(M) and the Digital Signature(DS) from A, it first uses the same message-digest algorithm as was used by A and calculates its own Message Digest (MD2) for M.



5. Now B uses A's public key to decrypt the digital signature because it was encrypted by A's private key. The result of this process is the original Message Digest (MD1) which was calculated by A.



6. If $MD1 == MD2$, the following facts are established as follows.
- *B accepts the original message M as the correct, unaltered message from A.*
 - *It also ensures that the message came from A and not someone posing as A.*



The message digest (MD1) was encrypted using A's private key to produce a digital signature. Therefore, the digital signature can be decrypted using A's public key (due to asymmetric form of RSA). If the receiver B is able to decrypt the digital signature using A's public key, it means that the message is received from A itself and now A cannot deny that he/she has not sent the message. It also proves that the original message did not tamper because when the receiver B tried to find its own message digest MD2, it matched with that of A's MD1. Suppose a malicious user tries to access the original message and perform some alteration.

Now he/she will calculate a new message digest over the altered message. It might concern you with data integrity and confidentiality but here's the catch. The attacker will have to sign the altered message using A's private key in order to pose as A for the receiver B. However, an attacker cannot sign the message with A's private key because it is known to A only. Hence, the RSA signature is quite strong, secure, and reliable.

Process of Creating a Digital Signature

The process of creating a digital signature involves the following steps:

1. **Hashing:** The first step in creating a digital signature is to create a hash of the message or document that needs to be signed. This is done using a hash function, which produces a fixed-length output (the hash value) from an input of any size.
2. **Signing:** The hash value is then encrypted using the private key of the signer. This produces the digital signature, which is attached to the original message or document.
3. **Verification:** To verify the authenticity of the digital signature, the recipient of the message or document must first decrypt the signature using the public key of the signer. This produces the original hash

value. The recipient then calculates the hash value of the received message or document using the same hash function that was used by the signer. If the two hash values match, the signature is valid and the message or document has not been tampered with.

COMMON RSA WEAKNESSES:

- **Small public exponent attacks** — Sometimes systems use small values like: $e = 3$. This can allow attackers to exploit mathematical shortcuts, especially when padding is not used.
 - *Basically, using very small exponents can reduce the complexity of breaking the encryption.*
- **Weak Or Reused Primes** — If the same prime numbers are reused in multiple keys, attackers may detect patterns. Also, weak random number generators may produce predictable primes.
 - *Basically, if primes are poorly generated, attackers can factor the modulus and break the key.*
- **Improper padding** — RSA should not encrypt raw data directly.
 - **Padding schemes such as:**
 - OAEP (Optimal Asymmetric Encryption Padding)
 - PKCS#1add randomness and prevent attacks.
 - **Without padding, RSA is vulnerable to:**
 - chosen plaintext attacks
 - message manipulation
 - *Basically, padding adds randomness and protects RSA from mathematical attacks.*

KEY EXCHANGE MECHANISMS:

— Key exchange protocols allow two parties to **securely agree on a shared secret key** over an insecure channel.

1. Diffie–Hellman Key Exchange

- Diffie–Hellman key exchange is a method of digital encryption that securely exchanges cryptographic keys between two parties over a public channel without their conversation being transmitted over the internet. The two parties use symmetric cryptography to encrypt and decrypt their messages. Published in 1976 by Whitfield Diffie and Martin Hellman, it was one of the first practical examples of public key cryptography.
- ***It is a protocol that allows two users to establish a shared secret.***
- It relies on the **difficulty of the discrete logarithm problem**.

How does it work?

- 1) Public parameters are chosen:
 - large prime p
 - generator g
- 2) Alice chooses secret a
- 3) Bob chooses secret b
- 4) They compute: $A = g^a \bmod p$
 $B = g^b \bmod p$
- 5) They exchange A and B .
- 6) Shared secret:
 - Alice computes: $S = B^a \bmod p$
 - Bob computes: $S = A^b \bmod p$

Both obtain the same value.

Basically, Diffie–Hellman lets two people create a shared secret without sending the secret itself.

2. Elliptic Curve Diffie–Hellman (ECDH)

- Elliptic-curve Diffie–Hellman (ECDH) is a key agreement algorithm that allows users to calculate a secret key using the public-private key pair and the generator point on the elliptic curve being used. This secret key allows both parties to communicate securely over an insecure channel.

- ECDH is a variant of the Diffie-Hellman algorithm that is based on **elliptic curves**. It is very similar to the classical Diffie-Hellman algorithm being used to exchange keys securely. ECDH, instead of doing exponential of keys like Diffie-Hellman, uses elliptic curve cryptography multiplication to calculate the keys.
- **ADVANTAGES:**
 - smaller keys
 - faster computation
 - stronger security per bit
- EXAMPLE: 256-bit ECC $\approx$ 3072-bit RSA security
*Basically, it is a **more efficient version of Diffie–Hellman using elliptic curves**.*
- This is widely used in **TLS, HTTPS, and secure messaging apps**.

DIGITAL CERTIFICATES

- Digital certificates solve the problem of trusting public keys.
- If someone sends a public key, how do you know it really belongs to them?
- Certificates provide that verification.

*Basically, a digital certificate is an **electronic identity card for a public key**.*

- **Certificate authorities (CA)** — are trusted, third-party entities that issue digital certificates (SSL/TLS) to verify the identity of websites, organizations, and individuals. They enable secure, encrypted, and trusted communication over the internet, forming the foundation of public key infrastructure (PKI).
 - EXAMPLES:
 - DigiCert
 - GlobalSign
 - Let's Encrypt
 - It verifies the identity of the entity and signs their certificate using its private key.

*Basically, the CA acts as a **trusted third party that confirms ownership of public keys**.*
- **Certificate Chains** — is an ordered list of digital certificates—starting from an end-entity (server) certificate, through intermediate CA certificates, and ending with a trusted root CA certificate. It verifies the authenticity of a certificate by linking it back to a trusted root authority.
 - Certificates often form a **chain of trust**.
 - **Structure:** Root CA $\rightarrow$ Intermediate CA $\rightarrow$ Server Certificate
 - **When a browser receives a certificate:**
 - It verifies the server certificate.
 - It checks the intermediate certificate.
 - It verifies the root certificate stored in the OS or browser.

*Basically, certificate chains allow trust to be **distributed across multiple levels of authorities**.*
- **Trust Models** — are frameworks defining rules, relationships, and mechanisms—such as Public Key Infrastructure (PKI) or reputation systems—used to validate identity, security, and reliability in digital, organizational, and IoT environments.
 - Different systems use different trust models.
 - **Key types** include **hierarchical (root CA)**, **bridge (peer-to-peer)**, and **Zero Trust (verification-based) models**, essential for secure, authorized, and trusted interactions.